Contents

Chapter 12: Performance and Economics

AI engineering is systems engineering under an unusual constraint:

> **The most expensive component in the system is often the component doing the reasoning.**

In a conventional web application, adding another API call might cost essentially nothing relative to the rest of the infrastructure.

In an AI application, every additional model invocation can consume:

- tokens
- GPU cycles
- memory bandwidth
- latency budget
- provider quota
- money

An agent that takes ten model calls instead of one is not merely "more intelligent."

It may be **10x more expensive, 10x slower, and 10x harder to scale**.

This makes performance and economics architectural concerns.

The fundamental optimization problem is:

$$\text{maximize useful work}$$

subject to:

$$\text{latency} \leq L_{max}$$

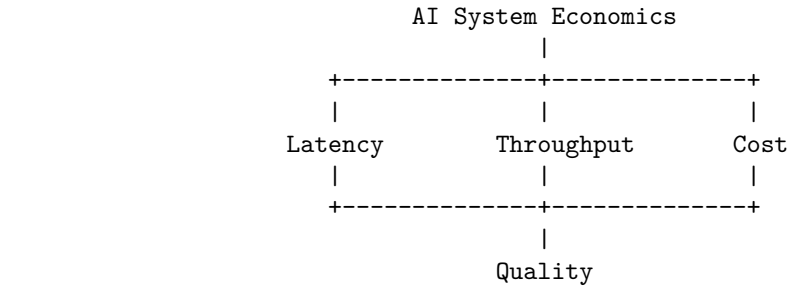
$$\text{cost} \leq C_{max}$$

$$\text{quality} \geq Q_{min}$$

and:

$$\text{throughput} \geq R_{required}$$

The engineer therefore needs to reason simultaneously about:



Optimizing any one dimension in isolation can make the overall system worse.

A faster model that costs 5x more may be a bad optimization.

A cheaper model that produces 30% more retries may also be a bad optimization.

A larger context window may improve answer quality while destroying both latency and cost.

The goal is not to make AI systems "fast."

It is to make them **economically efficient systems that satisfy explicit performance and quality requirements**.

---

1. Start With a Cost Model

The simplest useful model is:

$$C = N_{requests} \times (T_{input}P_{input} + T_{output}P_{output})$$

where:

- $N_{requests}$ = number of model requests
- T_{input} = input tokens per request
- T_{output} = output tokens per request
- P_{input} = price per input token
- P_{output} = price per output token

This simple equation already exposes several optimization levers.

You can reduce cost by reducing:

```
number of requests
input tokens
output tokens
price per token
```

That gives us:

```
                  Cost
                    |
       +------------+------------+
       |            |            |
   Requests       Input        Output
       |          tokens        tokens
       |            |            |
   caching       context      generation
   batching      retrieval    max tokens
   routing       compression  structured output
```

For agentic systems, the equation should be expanded.

If a task requires k model calls:

$$C_{task} = \sum_{i=1}^{k}(T_{input,i}P_{input,i} + T_{output,i}P_{output,i})$$

Now agent architecture directly affects economics.

A workflow:

```
User
 ↓
LLM
 ↓
Answer
```

might require one model call.

An agent:

```
Planner
 ↓
Retriever
 ↓
LLM
 ↓
Tool
 ↓
LLM
 ↓
Verifier
 ↓
LLM
```

might require five or six.

The second system may be substantially better—but it must justify the additional cost.

---

2. Performance Is Multidimensional

"Performance" is not one number.

Important metrics include:

Latency How long does one request take?

$$L = t_{response} - t_{request}$$

Throughput

How much work can the system perform per unit time?

$$Throughput = \frac{requests}{second}$$

Concurrency

How many requests are being processed simultaneously?

Utilization How much of the available compute capacity is actually being used?

Cost How much does each request or task consume?

Quality How useful or correct is the result?

A system can be:

```
fast but expensive
cheap but slow
high-throughput but low-quality
high-quality but impossible to scale
```

The engineering task is to find the appropriate point in this multidimensional space.

---

3. Latency

For an AI application, end-to-end latency can be decomposed as:

$$L_{total} = L_{network} + L_{retrieval} + L_{model} + L_{tools} + L_{postprocess}$$

For an agentic system:

$$L_{total} = \sum_{i=1}^{k} L_i$$

for sequential operations.

Consider:

```
User request
   ↓
Retrieval       100 ms
   ↓
LLM             2.0 s
   ↓
Tool            500 ms
   ↓
LLM             2.0 s
   ↓
Response
```

Total latency is approximately:

$$4.6s$$

If the operations can safely execute in parallel:

```
           +-- Retrieval --+
User ------|               +-- LLM
           +-- Tool -------+
```

the critical path may be much shorter.

This produces a fundamental optimization principle:

> **Optimize the critical path, not the average component.**

---

4. Tail Latency

Average latency is often misleading.

Suppose:

```
Average: 1.2 sec
P50:     0.8 sec
P95:     3.5 sec
P99:     12 sec
```

Most users experience a fast system.

But 1% of requests take 12 seconds.

At 100,000 requests per day, that is:

$$1,000$$

very slow requests every day.

AI systems often have significant tail latency because of:

- variable generation lengths
- queueing
- provider load
- retrieval variability
- tool failures
- retries
- long contexts
- agent step count

You should therefore measure:

```
P50
P90
P95
P99
P99.9
```

rather than relying on the mean.

For interactive systems, p95 or p99 latency may be much more relevant than average latency.

---

5. Time to First Token vs Time to Complete

Streaming introduces another useful distinction.

For an LLM request:

```
Request
   |
   +---- Time to First Token
   |
   +--------------- Time to Complete
```

Users often perceive the system as responsive once generation begins.

Therefore:

$$TTFT = t_{first\ token} - t_{request}$$

can matter independently of total generation time.

A system might have:

```
TTFT = 500 ms
Completion = 5 sec
```

and feel more responsive than one with:

```
TTFT = 3 sec
Completion = 4 sec
```

even though total latency is similar.

For conversational systems, optimizing TTFT can therefore provide significant UX improvements.

---

6. Throughput

Latency asks:

> How quickly does one request complete?

Throughput asks:

> How many requests can the system handle?

For an inference system:

$$Throughput = \frac{tokens}{second}$$

may be more useful than requests/second.

A model generating:

```
100 tokens/sec
```

can support very different workloads depending on average output length.

Similarly, input processing and output generation have different computational characteristics.

An inference server may therefore track:

```
input tokens/sec
output tokens/sec
requests/sec
concurrent sequences
GPU utilization
memory utilization
```

These measurements expose where capacity is actually going.

---

7. Batching

GPU hardware is optimized for parallel computation.

Running one request at a time often leaves substantial capacity unused.

Batching combines multiple requests:

```
Request 1 -+
Request 2 -+--> Batch --> GPU
Request 3 -+
```

Instead of:

```
GPU
 ↓
Request 1

GPU
 ↓
Request 2

GPU
 ↓
Request 3
```

batching can improve hardware utilization and throughput.

But batching introduces a tradeoff.

A request may wait for other requests to arrive.

Therefore:

$$Latency \uparrow$$

while:

$$Throughput \uparrow$$

The optimal batch size depends on workload characteristics.

For interactive applications, small dynamic batches may provide a better tradeoff.

For offline processing, large batches may be ideal.

---

8. Continuous Batching

Modern inference systems often use **continuous batching**.

Instead of waiting for an entire batch to finish:

```
Batch
 +-- Request A
 +-- Request B
 +-- Request C

wait for all
```

new requests can enter while existing requests are still generating.

Conceptually:

```
Time →
------------------------------>

A ###########
B #######
C #############
D       ########
E           #########
```

This allows the system to keep GPU resources busy despite different generation lengths.

For high-throughput inference, batching strategy can have a major effect on economics.

---

9. Concurrency

Concurrency is not the same as throughput.

A system might allow:

```
100 concurrent requests
```

but still process them inefficiently.

Increasing concurrency can improve utilization until some resource saturates.

After that point:

```
Concurrency ↑
     ↓
Queueing ↑
     ↓
Latency ↑
     ↓
Timeouts ↑
     ↓
Retries ↑
     ↓
Effective throughput ↓
```

This is a classic distributed-systems failure mode.

The goal is therefore not:

> "Maximize concurrency."

It is:

> **Find the concurrency level that maximizes useful throughput while satisfying latency and reliability constraints.**

---

10. Context Length Is a Performance Variable

A common mistake is to think of context merely as an AI-quality parameter.

It is also a systems parameter.

Consider:

```
Request A:
2,000 input tokens

Request B:
20,000 input tokens
```

Even if both generate 500 output tokens, Request B requires substantially more input processing.

Long contexts also consume more memory.

For transformer inference, attention historically had approximately quadratic scaling in sequence length:

$$O(n^2)$$

although modern architectures and inference optimizations can significantly change practical behavior.

The key engineering principle remains:

> **More context is not free.**

Every additional token can affect:

- latency
- memory
- throughput
- cost
- attention quality
- cache efficiency

Context engineering is therefore also performance engineering.

---

11. Context Compression

Suppose an agent accumulates:

```
Conversation
+ retrieved documents
+ tool outputs
+ intermediate results
+ previous responses
```

Instead of sending everything back to the model, the system can compress state:

```
Raw history
     ↓
Relevance filtering
     ↓
Summarization
     ↓
Structured state
     ↓
LLM
```

For example:

```
10,000 tokens
     ↓
2,000-token summary
```

If quality remains acceptable, the cost reduction can be substantial.

But compression is not automatically beneficial.

You must evaluate:

$$Quality_{compressed}$$

against:

$$Quality_{full}$$

The optimization objective is:

$$\min Cost$$

subject to:

$$Quality \geq Q_{min}$$

---

12. Caching

Caching is one of the highest-leverage performance optimizations.

Suppose 30% of requests are identical or semantically equivalent.

Without caching:

```
request
 ↓
LLM
```

With caching:

```
request
 ↓
cache lookup
 +-- hit → response
 +-- miss → LLM
```

The effective model workload becomes:

$$N_{LLM} = N_{requests}(1 - H)$$

where H is cache hit rate.

At:

$$H = 0.30$$

the model workload falls by 30%.

Caching can apply to:

- exact responses
- embeddings
- retrieval results
- tool results
- expensive computations
- intermediate agent states

But AI caching introduces semantic challenges.

If the user asks:

> "What is today's stock price?"

a cached response may be dangerous.

Therefore cache policy must consider:

- freshness
- personalization
- authorization
- semantic equivalence
- invalidation
- time-to-live

A cached answer is still subject to the same security and correctness requirements as a generated answer.

---

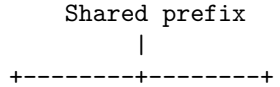
13. Prefix and Prompt Caching

Many AI requests share large common prefixes:

```
system instructions
+
tool definitions
+
policy
+
long document
+
user question
```

If the infrastructure supports prefix caching, repeated requests can reuse computation associated with the shared prefix.

Conceptually:



```
        |                   |
  Question A          Question B
```

This can reduce both latency and cost.

The general principle is:

> **Do not repeatedly pay for computation that has not changed.**

This is especially important for agent systems with large, stable system prompts or tool catalogs.

---

14. Model Routing

One of the most powerful AI-specific optimizations is model routing.

Instead of sending every task to the same model:

```
All requests
     ↓
Model X
```

use:

```
                  +-- cheap model
                  |
Request → Router  +-- medium model
                  |
                  +-- expensive model
```

The router considers:

- task difficulty
- latency requirement
- quality requirement
- context length
- user tier
- cost budget
- model availability

This produces an important economic principle:

> **Use the cheapest model that can reliably solve the task.**

---

15. Model Routing Example

Suppose we have two models.

Model A

```
cheap
slow
high quality
```

Model B

```
expensive
fast
high quality
```

A naive architecture might always use Model B.

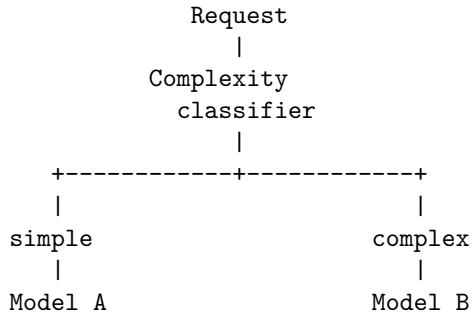
That guarantees low latency but maximizes cost.

Another naive architecture might always use Model A.

That minimizes cost but may violate latency SLOs.

A router can divide the workload.

For example:



But "simple" and "complex" are not enough.

Routing should ideally optimize a utility function.

For example:

$$U_i = \alpha Q_i - \beta L_i - \gamma C_i$$

where:

- Q_i = expected quality
- L_i = expected latency
- C_i = expected cost

The router chooses:

$$i^* = \arg\max_i U_i$$

subject to hard constraints such as:

$$Q_i \geq Q_{min}$$

and:

$$L_i \leq L_{max}$$

---

16. Difficulty-Aware Routing

The simplest routing strategy uses heuristics.

For example:

```
Short classification
        ↓
Cheap model

Simple extraction
        ↓
Cheap model

Complex reasoning
        ↓
Powerful model

High-risk operation
        ↓
Best available model + verification
```

A more sophisticated router can estimate task difficulty.

Possible signals include:

- input length
- number of retrieved documents
- task type
- previous model confidence
- number of required tools
- historical failure rate
- user requirements

The routing architecture becomes:

```
Request
   ↓
Feature extraction
   ↓
Difficulty / risk estimate
   ↓
Model selection
   ↓
Execution
```

```
   ↓
Evaluation
   ↓
Optional escalation
```

This last step is important.

A cheap model does not need to solve every difficult problem.

It can attempt the task and escalate when necessary:

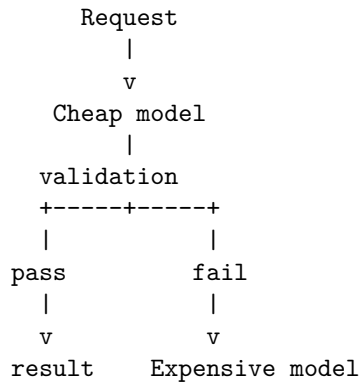
```
Cheap model
     ↓
Validation
 +---+----+
 |        |
pass     fail
 |        |
done   expensive model
```

This can dramatically improve economics while preserving quality.

---

17. Cascade Architectures

Model routing can be implemented as a cascade.



Suppose:

- 80% of tasks succeed with Model A
- 20% require Model B

Then expected cost is:

$$E[C] = 0.8C_A + 0.2(C_A + C_B)$$

or:

$$E[C] = C_A + 0.2C_B$$

assuming Model A runs first on every request.

If C_A is very small, this can be substantially cheaper than sending everything directly to Model B.

But the additional Model A latency must also be considered.

Again:

Cost optimization is constrained by latency and quality.

---

18. Quantization

For self-hosted models, quantization can dramatically change the economics.

A model's parameters may be represented using:

```
FP32
FP16
BF16
INT8
INT4
```

Reducing precision generally reduces:

- memory footprint
- memory bandwidth requirements
- storage
- potentially inference cost

For example, a parameter represented using 16 bits requires half the raw parameter storage of a 32-bit representation.

A rough relationship is:

$$Memory \approx N_{parameters} \times \frac{bits}{8}$$

before accounting for additional runtime memory such as:

- KV cache
- activations
- temporary buffers
- framework overhead

Quantization therefore enables larger models to fit into available hardware.

But it introduces a quality tradeoff.

The correct question is not:

> "Is INT4 better than FP16?"

It is:

> **"How much quality do we lose per unit of memory or throughput gained?"**

That must be measured empirically.

---

19. KV Cache and Memory

For autoregressive transformer inference, the KV cache stores intermediate attention state for previously processed tokens.

As context length and concurrency increase, KV-cache memory can become a major constraint.

Conceptually:

```
Model weights
+
KV cache
+
activations
+
runtime buffers
=
GPU memory
```

This creates an important systems tradeoff.

A larger context window may not simply make one request more expensive.

It may reduce the number of simultaneous sequences that fit in memory.

Therefore:

```
context length ^
      ↓
KV cache ^
      ↓
concurrency capacity ↓
      ↓
throughput ↓
```

This is why model serving requires thinking about **memory capacity and bandwidth**, not just FLOPs.

---

20. GPU Utilization

A GPU that is only 20% utilized may indicate that inference is inefficient.

But high utilization does not automatically mean the system is optimal.

You need to distinguish:

```
compute-bound
memory-bound
communication-bound
latency-bound
```

For inference workloads, memory bandwidth can be particularly important.

A model may spend significant time moving parameters and KV-cache data rather than performing arithmetic.

Useful metrics include:

```
GPU utilization
memory utilization
memory bandwidth
tokens/sec
requests/sec
batch size
KV-cache occupancy
power
```

This is where AI engineering begins to look very much like traditional performance engineering.

The optimization loop is familiar:

```
Measure
  ↓
Profile
  ↓
Identify bottleneck
  ↓
Change one variable
  ↓
Benchmark
  ↓
Compare
  ↓
Repeat
```

Never optimize based solely on intuition.

---

21. Bottleneck Analysis

Suppose your AI service has:

```
CPU utilization:       20%
GPU utilization:       95%
Memory utilization:    60%
Network utilization:   10%
```

The GPU is probably the bottleneck.

But consider another workload:

```
CPU utilization:       80%
GPU utilization:       30%
Memory bandwidth:      95%
```

Now the problem may be memory movement or preprocessing.

Or:

```
GPU utilization:       40%
LLM latency:           2 sec
queueing latency:      8 sec
```

The GPU is not the bottleneck.

The queue is.

The general rule is:

> **Measure the critical resource before optimizing it.**

---

22. Parallelism

Agentic workflows often contain unnecessary sequential dependencies.

Consider:

```
Question
   ↓
Search A
   ↓
Search B
   ↓
Search C
   ↓
LLM
```

If the searches are independent:

```
             +-- Search A --+
Question ----+-- Search B --+-- LLM
             +-- Search C --+
```

the total latency becomes approximately:

$$L = \max(L_A, L_B, L_C) + L_{LLM}$$

rather than:

$$L = L_A + L_B + L_C + L_{LLM}$$

This can provide dramatic improvements.

The key question is:

Which operations actually depend on each other?

The agent graph should encode those dependencies explicitly.

---

23. Speculative and Early-Exit Strategies

Performance can sometimes be improved by doing inexpensive work before expensive work.

For example:

```
Request
   ↓
Cheap classifier
   ↓
Does this require an LLM?
 +-+-+
No  Yes
 |    |
rule  LLM
```

Many production workloads contain requests that can be handled without invoking a large model.

Examples:

- deterministic validation
- simple classification
- cached responses
- authentication failures
- known commands
- FAQ lookup
- structured transformations

The best model call is often:

the model call you never make.

---

24. Economics of Agentic Workflows

Consider three architectures.

Architecture A

```
1 model call
```

Architecture B

```
planner
 ↓
tool
 ↓
model
```

Architecture C

```
planner
 ↓
retrieval
 ↓
model
 ↓
tool
 ↓
model
 ↓
verification
 ↓
model
```

Suppose the cost of one model call is C.

Then approximately:

$$C_A = C$$

$$C_B = 2C$$

$$C_C = 4C$$

before accounting for different token volumes.

The additional calls may improve quality.

But every call should have an engineering justification.

A useful metric is:

$$Value\ per\ dollar = \frac{Task\ Quality}{Inference\ Cost}$$

Another is:

$$Value\ per\ second = \frac{Task\ Quality}{Latency}$$

The optimal architecture depends on the application.

---

25. Cost Per Successful Task

Raw cost per request can be misleading.

Suppose:

```
Model A
Cost = $0.01
Success rate = 80%

Model B
Cost = $0.05
Success rate = 98%
```

The cost per successful task is approximately:

$$\frac{0.01}{0.80} = \$0.0125$$

for A and:

$$\frac{0.05}{0.98} \approx \$0.051$$

for B.

Model A is still substantially cheaper per successful task.

But suppose failed requests trigger retries.

If Model A requires expensive fallback processing, its economics can change dramatically.

Therefore measure:

$$Cost_{successful\ task}$$

rather than only:

$$Cost_{request}$$

This is especially important for routing and agent systems.

---

26. The Full Cost Model

A realistic production cost model should include more than model tokens.

For example:

$$C_{total} = C_{inference} + C_{retrieval} + C_{storage} + C_{network} + C_{compute} + C_{observability} + C_{failed\ work}$$

For self-hosted inference:

$$C_{inference} \approx C_{GPU} + C_{power} + C_{memory} + C_{host} + C_{operations}$$

The economic objective becomes:

$$C_{task} = \frac{C_{infrastructure}}{N_{successful\ tasks}}$$

This is much closer to the real economics of an AI product.

---

27. Optimization Order

When optimizing an AI application, use a disciplined sequence.

First: eliminate unnecessary work

```
Do we need this model call?
Do we need this tool call?
Do we need this context?
Do we need this verification step?
```

Second: reduce data

```
Can retrieval return fewer documents?
Can context be compressed?
Can outputs be shorter?
```

Third: reuse work

```
Can we cache it?
Can we reuse embeddings?
Can we cache prefixes?
```

Fourth: parallelize

```
Can independent operations execute concurrently?
```

Fifth: route intelligently

```
Can a smaller model handle this task?
```

Sixth: optimize inference

```
batching
quantization
GPU utilization
memory bandwidth
KV cache
```

This order matters.

It is usually better to eliminate 50% of unnecessary model calls than to optimize the remaining calls by 10%.

---

28. The Performance Experiment

Take the Week 1 Personal Research Assistant.

Measure:

```
requests/sec
P50 latency
P95 latency
P99 latency
TTFT
input tokens/request
output tokens/request
model calls/task
GPU utilization
cache hit rate
cost/request
cost/successful task
```

Then establish a baseline.

For example:

```
Baseline
--------------
P95 latency       8.2 s
Input tokens      8,000
Output tokens     900
Model calls       3.2
Cache hit rate    0%
Cost/task         $0.12
```

Now optimize one dimension at a time.

Experiment 1 — Context reduction Reduce:

```
8,000 → 4,000 input tokens
```

Measure:

- quality
- latency
- cost

Experiment 2 — Caching Add retrieval caching.

Measure:

```
cache hit rate
LLM calls avoided
latency reduction
cost reduction
```

Experiment 3 — Parallel retrieval Run independent retrieval operations concurrently.

Measure:

```
P95 latency
```

Experiment 4 — Model routing Send simple requests to Model A.

Measure:

```
quality
latency
cost
fallback rate
```

Experiment 5 — Batching Increase concurrency and batch size.

Measure:

```
tokens/sec
requests/sec
GPU utilization
P95 latency
```

The objective is not to produce the most impressive benchmark.

It is to understand **which engineering changes actually move the system's economic frontier**.

---

29. Build the Routing Strategy

For today's exercise, start with the two-model scenario.

Model A

```
Cheap
Slow
High quality
```

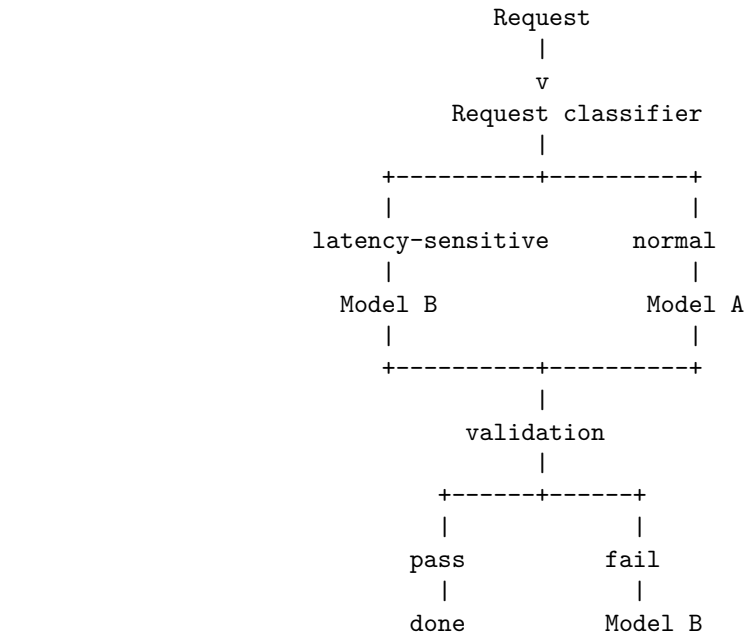
Model B

```
Expensive
Fast
High quality
```

Define:

```
quality threshold
latency threshold
cost budget
```

Then construct:



Then measure the economics.

Let:

$$p = P(\text{Model A succeeds})$$

and let:

$$C_A, C_B$$

be the respective costs.

The expected cost of the cascade is:

$$E[C] = C_A + (1 - p)C_B$$

Compare this with:

$$C_B$$

for sending every request directly to Model B.

Now include latency:

$$E[L] = L_A + (1 - p)L_B$$

for a sequential fallback architecture.

If this violates your latency SLO, consider:

- routing directly to B for latency-sensitive tasks
- improving the classifier
- using a faster cheap model
- parallel speculative execution
- reducing context
- caching

This is the kind of reasoning expected from an AI systems engineer.

---

30. The Performance Mindset

The naive question is:

> "Which model is best?"

The systems engineer asks:

> "Best according to what objective function?"

Then:

> "What is the cost per successful task?"

Then:

> "What is the critical path?"

Then:

> "Where is the bottleneck?"

Then:

> "Can we eliminate this computation?"

Then:

> "Can we cache it?"

Then:

> "Can we parallelize it?"

Then:

> "Can a smaller model do it?"

Then:

> "Can we trade memory for compute, or compute for memory?"

And finally:

> **"What is the cheapest architecture that satisfies the quality, latency, throughput, and reliability requirements?"**

That is performance engineering.

AI does not change these fundamentals.

It makes them more economically consequential.

---

31. Key Takeaways

1. **Performance and economics are architectural concerns.** Model calls consume real latency, compute, memory, quota, and money.
2. **Start with an explicit cost model.** The basic model is:

$$C = N_{requests}(T_{input}P_{input} + T_{output}P_{output})$$

3. **Agentic systems multiply cost through repeated model calls.** Optimize the number of calls before optimizing individual calls.
4. **Measure the complete performance profile.** Track latency, TTFT, throughput, concurrency, utilization, tokens, and cost—not a single benchmark number.
5. **Optimize the critical path.** Independent retrievals and tool calls should execute concurrently whenever correctness permits.
6. **Tail latency matters.** P95 and P99 often describe production experience better than averages.
7. **Context is both a quality and performance resource.** More context increases computation, memory consumption, latency, and often cost.
8. **Caching can be one of the highest-leverage optimizations.** Cache responses, retrieval, embeddings, tool results, and reusable computation when correctness and freshness permit.

9. **The best model call is often the one you never make.** Use deterministic logic, caching, retrieval, and lightweight classifiers to eliminate unnecessary inference.

10. **Batching improves hardware efficiency but can increase latency.** Optimize batch size according to workload requirements.

11. **GPU utilization must be interpreted alongside memory bandwidth and KV-cache behavior.** High utilization is not itself proof of an efficient system.

12. **Quantization is an economic optimization.** Lower precision can reduce memory requirements and improve inference efficiency, but quality degradation must be measured.

13. **Model routing changes the economics of AI systems.** Use the cheapest model that reliably satisfies the task's quality and latency constraints.

14. **Cascade architectures can combine cheap and expensive models.** Let inexpensive models handle easy cases and escalate difficult or failed cases.

15. **Measure cost per successful task, not merely cost per request.** Retries, failures, and fallback models can radically change the real economics.

16. **Optimize in the right order:**

```
eliminate work
    ↓
reduce data
    ↓
cache
    ↓
parallelize
    ↓
route
    ↓
optimize inference
```

17. **Performance optimization is empirical.** Measure → profile → change → benchmark → compare.

18. **The ultimate objective is not "maximum speed" or "minimum cost."** It is:

Maximum useful capability	subject to	quality, latency, reliability, and cost constraints

The central lesson is that an AI engineer should think of every token, model invocation, GPU cycle, byte of context, and millisecond of latency as a **resource allocation decision**.

The best AI system is not necessarily the one with the most powerful model.

It is the one that uses **exactly as much intelligence as the task requires—and no more**.